Keep It Warm: Context Trimming Raises the Cost of Tool-Using LLM Agents Under Prefix-Cache Pricing

Vatsal Roy
Independent project
Code and data: https://github.com/VatsalRoy/keep-it-warm

***Abstract*—Long-running LLM agents resend their growing history on every turn, and practitioners routinely trim that history to save money. Inference servers, however, cache the *prefix* of a prompt: any edit to an earlier message forces everything after it to be recomputed at the full price. We measure the net effect of six client-side context policies on a reproducible, exact-answer, tool-using benchmark (48 questions in 12 four-question sessions), served by vLLM with prefix caching on commodity GPUs, and price every request under two cache-discount regimes. Every trimming policy reduced input tokens by 22–46%, yet under a 10× cache discount the policies that edit the prompt frequently cost 1.51–2.62× more per solved question than keeping the full history (95% bootstrap intervals exclude 1.0). Batching the same edits reduced the penalty, and editing less often reduced it further. A single timestamp at the top of the system prompt removed all cache hits and raised the cost per solved question 6.6×, more than any policy choice. Threshold summarization was the only policy statistically tied with full history on cost while also lowering tail latency. All policies scored lower on accuracy than full history (by 6–12 points), though no difference was significant; we show that GPU non-determinism at temperature 0 alone flips 6 of 48 answers between identical runs.**

***Index Terms*—LLM agents, prompt caching, KV cache, context management, inference cost, vLLM, benchmarking**

I. Introduction

An LLM agent that uses tools (searching logs, reading files, calling APIs) accumulates a history of actions and tool outputs. Because the model is stateless, the entire history is sent again on every turn, so input cost grows roughly quadratically with the number of turns. A common response is *context engineering*: dropping old turns, masking old tool outputs, or summarizing the history once it grows large [2], [6], [7].

Modern serving stacks reduce the cost of repeated input with *prefix caching*: the attention key/value (KV) states computed for a prompt are stored in blocks and reused when a later prompt starts with exactly the same tokens [3]–[5]. Providers pass this on as a discounted price for cached input. The interaction between the two practices is easy to miss: a trimming policy that edits an *early* part of the history invalidates the cached prefix from that point on, so a shorter prompt can contain more *uncached* tokens than a longer one. Recent work quantifies this effect and proposes editing the KV cache on the server instead of the text [1].

TABLE I
Prefix-cache smoke test (vLLM 0.32.0, Qwen3-4B, T4)

Request	Prompt	Cached	TTFT
1. New prefix	5,555	0	14.4 s
2. Same prefix, new question	5,553	5,520	0.23 s
3. One char. changed at start	5,556	0	13.4 s

This paper asks the practitioner's version of the question: using only client-side policies on a stock server, which context policy gives the best accuracy per dollar once cache pricing is counted, and how much depends on *where* and *how often* the context is edited? Our contributions are:

- a seeded, exact-answer, long-horizon tool-use benchmark with a tool-only oracle that proves every task solvable, and per-request cache telemetry including the first token position at which each prompt changed;
- a controlled comparison of six context policies under two price sheets, with session-level paired bootstrap confidence intervals;
- evidence that frequent prompt edits raise cost per solved question by 1.5–2.6× under steep cache discounts, that edit cadence and prompt hygiene dominate, and a measured noise floor for accuracy at temperature 0.

II. Background

A. Prefix caching

vLLM's automatic prefix caching splits a prompt into fixed-size blocks (16 tokens here) and identifies each block by a hash of its tokens *and all preceding blocks* [3]. A request reuses cached blocks only up to the first block that differs. Changing one token near the start therefore invalidates every block after it, regardless of how much of the rest is identical. Table I reproduces this on our setup: an identical 5.5K-token prefix was 99.4% cached and its first token arrived 60× sooner, while a single changed character removed every hit.

B. Cost model

For a request with u uncached input tokens, c cached input tokens and o output tokens, the cost is

$$\text{cost} = u\,p_{\text{miss}} + c\,p_{\text{hit}} + o\,p_{\text{out}}. \quad (1)$$

The *naive* estimate ignores the cache and prices all input at p_{miss}. We use two sheets (USD per million tokens): **A**, p_{miss}=0.15, p_{hit}=0.075, p_{out}=0.60 (2× discount, as in [1]); and **B**, p_{miss}=5.00, p_{hit}=0.50, p_{out}=25.00 (10× discount, premium-API style, as in [2]).

III. Benchmark and Agent

A. Incident Desk

A seeded generator builds a fictional company: 8 microservices, 5 days of logs (220–320 lines per service-day), a deploy history, an on-call schedule and runbooks. Signals are planted with distractors (e.g., a deploy to another service just before an out-of-memory event). Twenty-four tasks in three tiers have exact answers, graded by normalized string matching with no LLM judge. A scripted oracle using only the agent's tools solves all 24, and the world is byte-identical across machines for the same seed.

To obtain long-horizon contexts, tasks are grouped into 12 *sessions* of 4 questions answered in one conversation. Each task appears exactly twice, at mirrored positions (first/fourth, second/third), with different companions.

B. Agent

The agent is Qwen3-4B-Instruct-2507 [8] (fp16) on vLLM 0.32.0 with the Triton attention backend, on Kaggle Tesla T4 GPUs (one server per GPU, concurrency 1, maximum context 28,672 tokens). It replies with one JSON action per turn over four tools: `ls`, `read` (at most 60 lines), `grep` (one file) and `submit`. Decoding uses temperature 0 and at most 20 turns per question. The system prompt (about 750 tokens, including a data guide) was refined over two pilot runs and then frozen. The prefix cache is reset before every session.

C. Telemetry

For each request we log prompt, cached, uncached and output tokens (from the server's usage report, cross-checked against its prefix-cache counters), time to first token (TTFT), and `first_changed`, the index of the first token that differs from the previous prompt. A prompt that only appends has `first_changed` equal to the previous prompt's length.

IV. Context Policies

Each policy maps the full, append-only history to the messages sent on a turn:

- **P0 Full history**: send everything (append-only).
- **P1 Sliding window**: system prompt, all questions, and the last 6 steps.
- **P2 Rolling masking**: replace tool outputs older than the last 4 with a short note, keeping the agent's actions [6].
- **P3 Batched masking**: the P2 edit applied only every N turns (N=6; sweep 3, 6, 10).
- **P4 Threshold summary**: above 8K tokens, the model summarizes the history and continues from the system prompt, the summary and the current question. Summary calls are billed.
- **P5 Hygiene bug**: P0 plus a changing timestamp on the first line of the system prompt.

A unit test asserts, before any GPU run, where each policy changes the prompt: never for P0; every turn once full for P1; only on fold turns for P3; only after a summary for P4; within the first 20 tokens for P5.

V. Results

Table II reports the main results. Intervals come from 2,000 bootstrap resamples of the 12 sessions (the four questions of a session share one conversation), paired with P0 [9]. P4 pools two independent runs. P5 and the cadence sweep were run on 4-session subsets and are compared with P0 on the same sessions.

A. Token savings do not survive cache pricing

Table III contrasts the naive estimate with the true cost. Under sheet B, P1 and P2 cut input tokens by 46% and 23% but raised the true cost by 96% and 87%, because they edit the prompt near its start on almost every turn (cached share 37% and 61% vs. 93% for P0). Under sheet A the same token cuts translated into cost changes between -20% and $+1\%$. The sign of the saving therefore depends on the cache discount (Fig. 1).

B. Edit cadence and prompt hygiene

Applying the masking edit in batches (P3) lowered the sheet-B ratio from 2.62× to 1.51×, but did not reach parity with P0. On the four longest sessions, masking every 3, 6 and 10 turns changed true cost by $+61\%$, $+14\%$ and $+2\%$ (Fig. 2): fewer, larger edits keep more of the prefix cached. The timestamp bug (P5) removed every cache hit and raised the cost per solved question 6.6×, more than any policy choice. Fig. 3 shows the per-request breakdown for one session.

C. Latency

P4 had the lowest tail latency (p95 TTFT 6.0 s vs. 10.5 s for P0) because its prompts stay short, and it was the only policy statistically tied with P0 on cost (1.11×, CI [0.95, 1.33]). On the T4, prefill throughput fell from about 330 to 85 new tokens/s as the context grew from 1.6K to 13.4K tokens, so a cache miss late in a session costs more time than an early one.

D. Accuracy

Every policy scored below P0 (by 6–12 points) but no paired interval excluded zero (P2: -12 [-27, $+0$]). P0 itself lost 12 points on questions 3–4 relative to 1–2, and no policy lost significantly more, so the position hypothesis is not supported at this sample size.

Hand-reading the lost answers explains part of the gap. About a third of P2's losses were caused by masking: in

TABLE II
MAIN RESULTS: 12 SESSIONS, 48 QUESTIONS PER POLICY. COSTS PER SOLVED QUESTION IN USD; RATIOS VS. P0 WITH 95% BOOTSTRAP CIS.

Policy	Accuracy [CI]	Turns/q	Cached	Tokens vs P0	$/solved A	$/solved B	B vs P0 [CI]
P0 full history	44% [29, 58]	5.8	93%	—	0.0070	0.090	1.00×
P1 sliding window	38% [19, 56]	6.1	37%	−46%	0.0071	0.205	2.29× [1.64, 3.80]
P2 rolling masking	31% [15, 48]	7.0	61%	−23%	0.0100	0.235	2.62× [1.83, 5.12]
P3 batched masking	35% [19, 54]	6.4	84%	−22%	0.0077	0.136	1.51× [1.06, 2.57]
P4 summary (2 runs)	38% [25, 52]	6.1	91%	−26%	0.0066	0.100	1.11× [0.95, 1.33]
P5 timestamp (4 sess.)	44% vs 56%	—	0%	+24%	—	—	6.60×

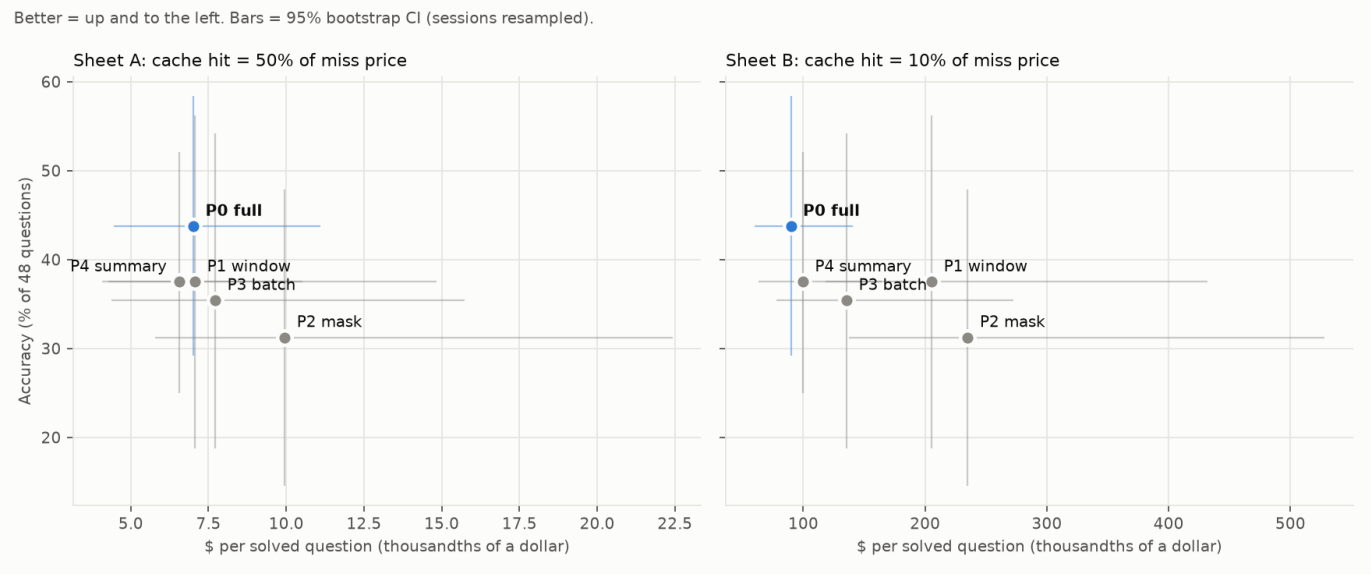


Fig. 1. Accuracy vs. cost per solved question under sheet A (left) and sheet B (right). Bars are 95% bootstrap CIs. Up and to the left is better.

TABLE III
NAIVE VS. TRUE COST CHANGE RELATIVE TO P0 (ONE RUN OF 12 SESSIONS)

Policy	Tokens	Naive	True A	True B
P1 window	−46%	−42%	−14%	+96%
P2 mask	−23%	−21%	+1%	+87%
P3 batch	−22%	−19%	−11%	+22%
P4 summary	−26%	−23%	−20%	−5%

one, the agent concluded "inventory has 7 … others have 0" after the counts of the other services, including the correct answer (19), had been masked. P2 also needed 1.27 more turns per question, re-running searches it could no longer see. Summaries recorded the agent's current beliefs as facts, for example a correct timestamp paired with the wrong deploy ID, after which the agent answered without re-checking.

E. Noise floor

Running P4 twice with identical code and temperature 0 changed the outcome of 6 of 48 questions (17 vs. 19 correct), and half of the sessions diverged, one at the very first action. GPU floating-point results are not bit-identical across runs, and near-tied next tokens amplify small differences. Total cost differed by only 2%. Cost conclusions are therefore robust, while accuracy differences of a few questions are within noise.

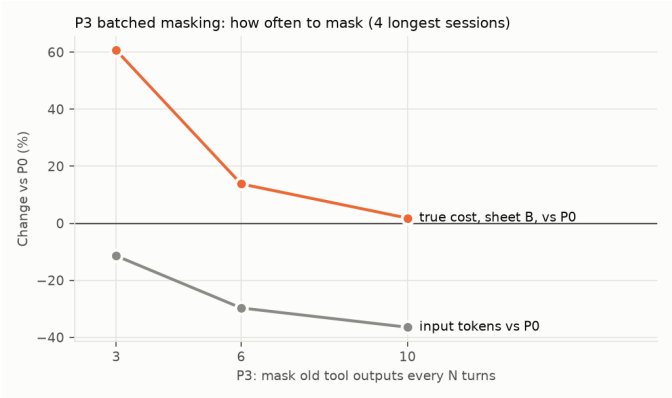


Fig. 2. Batched masking cadence on the four longest sessions: change in input tokens and in true cost (sheet B) relative to P0.

VI. DISCUSSION

Three practical rules follow. First, evaluate context policies on *uncached* input, not total tokens: with steep cache discounts, an append-only history is a strong baseline. Second,



Fig. 3. Cached (blue) and uncached (orange) input tokens per request in session s10. The sliding window re-reads most of its window on every request; the summary policy pays once at the fold.

if the history must be edited, edit rarely and in large steps, keeping the system prompt and tool definitions byte-identical. Third, audit prompt hygiene before tuning policies: a changing value at the top of a prompt can outweigh every other optimization. Server-side approaches that edit the KV cache directly [1] avoid the trade-off entirely; our results bound what client-side policies alone can achieve.

VII. Limitations

We study one 4B model, one synthetic domain and 48 questions per policy, so accuracy findings are suggestive. Requests ran at concurrency 1, so cache eviction under multi-user load was not tested. The two price sheets are illustrative; real providers add rules such as minimum cacheable lengths and cache lifetimes. P5 and the cadence sweep used four-session subsets. The P4 threshold (8K) was chosen below the original 16K because sessions peaked at about 11K tokens on average.

VIII. Conclusion

On a reproducible tool-use benchmark, every context-trimming policy reduced tokens, but those that edit the prompt frequently cost 1.5–2.6× more per solved question under a 10× cache discount, and a single timestamp line cost 6.6×. Counting tokens is not counting money: the position and frequency of edits, not their size, determine the bill.

REFERENCES

[1] Tsinghua NICS-EFC, "Cache engineering for agents," arXiv:2610.28577, Oct. 2026.

[2] UCSD, "Pack light, CarryOn," arXiv:2610.18784, Oct. 2026.

[3] vLLM Project, "Automatic prefix caching," vLLM documentation, 2026. [Online]. Available: https://docs.vllm.ai/en/stable/features/automatic_prefix_caching/

[4] W. Kwon *et al.*, "Efficient memory management for large language model serving with PagedAttention," in *Proc. ACM SOSP*, 2023.

[5] L. Zheng *et al.*, "SGLang: Efficient execution of structured language model programs," arXiv:2312.07104, 2023.

[6] "The complexity trap: Simple observation masking is as efficient as LLM summarization for agent context management," arXiv:2508.21433, 2025.

[7] "Masking stale observations helps search agents, until it doesn't," arXiv:2606.00408, 2026.

[8] Qwen Team, "Qwen3 technical report," arXiv:2505.09388, 2025.

[9] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. New York, NY, USA: Chapman & Hall, 1993.